

A Hybrid Simulated Annealing Algorithm for Nonslicing VLSI Floorplanning

Jianli Chen, Wenxing Zhu, and M. M. Ali

Abstract—Floorplanning in very large scale integrated-circuit (VLSI) design is the first phase in the process of designing the physical layout of a chip. This makes the floorplanning problem of paramount importance, since it determines the performance, size, yield, and reliability of VLSI chips [1]. From the computational point of view, the VLSI floorplanning is an NP-hard problem. In this paper, we present a hybrid simulated annealing algorithm (HSA) for nonslicing VLSI floorplanning. The HSA uses a new greedy method to construct an initial B*-tree, a new operation on the B*-tree to explore the search space, and a novel bias search strategy to balance global exploration and local exploitation. Experimental results on Microelectronic Center of North Carolina (MCNC) benchmarks [29] show that the HSA can quickly produce optimal or nearly optimal solutions for all the tested problems.

Index Terms—B*-tree, bias search strategy, local search, simulated annealing, very large scale integrated-circuit (VLSI) floorplanning.

I. INTRODUCTION

GIVEN a set of circuit components, or modules, and a net list specifying interconnections between the modules, the goal of very large scale integrated-circuit (VLSI) floorplanning is to find a floorplan for the modules such that no module overlaps with another, and the area of the floorplan and the interconnections between the modules are minimized. Floorplanning in VLSI design is the first phase in the process of designing the physical layout of a chip. This makes the floorplanning problem of paramount importance since it determines the performance, size, yield, and reliability of VLSI chips [1].

Two classes of floorplanning structures, slicing and nonslicing, have been identified [1]. Since most of the floorplannings are nonslicing [2], in this paper, we address the nonslicing floorplanning problem.

Representation of floorplans is important, since it determines the size of the search space and the complexity of transformation

between a representation and its corresponding floorplan. A lot of representations have been proposed for nonslicing floorplans [3]–[12]. A detailed comparison of these representations can be found in [13].

One of the most efficient nonslicing representations is the B*-tree representation proposed by Chang *et al.* [3]. Inheriting from the particular characteristics of the ordered binary tree, the B*-tree has many advantages compared with other representations [3]. First, the B*-tree representation is very fast and easy for implementation. In a B*-tree, there are not more than two branches at any node, so a B*-tree can be implemented by either a static data structure or a dynamic one. Using static memory, we can search and insert a node in $O(1)$ time while the O-tree representation needs $O(n)$ time [4]. Second, the B*-tree representation is very flexible for handling the floorplanning problem with various types of modules (e.g., hard, soft, pre-placed, and rectilinear modules) directly and efficiently. Third, in a B*-tree, we do not need to construct constraint graphs for area cost evaluation. Fourth, except for handling soft modules, it is easy to transform a B*-tree to its corresponding floorplan, which takes only linear time. Finally, like O-tree, the size of the search space of the floorplanning problem by the B*-tree representation is only $O(n!2^{2n-2}/n^{1.5})$ [3], while it is $O((n!)^2)$ by the sequence pair representation [8]. Hence, the B*-tree representation is adopted in this paper.

Since the VLSI floorplanning problem is NP-hard, various heuristic methods using different strategies have been proposed. They can be categorized into two classes: the constructive methods and the iterative methods.

The constructive methods use heuristic information to construct a floorplan. The bottom-left (BL) and BL fill methods [14], [15] are the most common constructive approaches. An improved BL heuristic method was presented by Liu *et al.* [16], and a best fit method was suggested by Burke *et al.* [17]. In the hierarchical approach proposed by Lengauer *et al.* [18], a hierarchy or cut-tree is generated initially in a top-down manner by a recursive min-cut method, and then the bottom levels of the tree are obtained by bottom-up clustering. Dong *et al.* [2] used the principle of less flexibility first to construct a floorplan. Huang *et al.* [19] presented a very effective heuristic algorithm, in which two important concepts, namely, the corner-occupying action and caving degree, were introduced to guide the packing process.

The iterative methods use metaheuristic strategies, such as simulated annealing, genetic algorithm, and tabu search, to produce good solutions. In [20], a genetic algorithm was applied to tackle the VLSI floorplanning problem. In [21], a memetic algorithm (MA) for the nonslicing and hard-module VLSI

Manuscript received September 21, 2009; revised January 27, 2010 and June 5, 2010; accepted July 30, 2010. Date of publication September 13, 2010; date of current version June 17, 2011. This work was supported in part by the National Science Foundation of China under Grant 60773126, in part by the National Key Basic Research Special Foundation (NKBRSF) of China under Grant 2011CB808000, and in part by the Research Fund for the Doctoral Program (RFDP) of China under Grant 20093514110004. This paper was recommended by Associate Editor J. Lazansky.

J. Chen and W. Zhu are with the Center for Discrete Mathematics and Theoretical Computer Science, Fuzhou University, Fuzhou 350002, China (e-mail: n70320076@fzu.edu.cn; wxzhu@fzu.edu.cn).

M. M. Ali is with the School of Computational and Applied Mathematics, Witwatersrand University, Wits 2050, Johannesburg, South Africa (e-mail: Montaz.Ali@wits.ac.za).

Digital Object Identifier 10.1109/TSMCC.2010.2066560

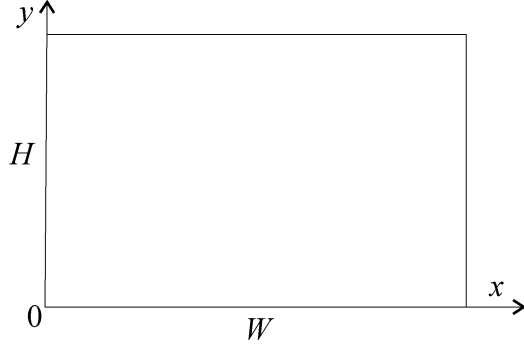


Fig. 1. Region P.

floorplanning problem was presented. This MA is a hybrid genetic algorithm which uses an effective genetic search method to explore the search space and an efficient local search method to exploit information in the search region.

Simulated annealing is one of the major techniques to solve the VLSI floorplanning problem [1]. Timber Wolf in [22] presented one of the first floorplanning systems based on the simulated annealing technique, which produces not only the relative positions of the modules, but also their aspect ratios and pin positions. The algorithms reported in [23] and [24] also are based on the simulated annealing. The algorithm presented in [23] can handle slicing structures with rectangular modules only, whereas the algorithm in [24] can produce nonslicing floorplans and even L-shaped modules. Chang *et al.* in [3] proposed a simulated annealing method based on the B*-tree representation.

In this paper, we develop a new heuristic algorithm for the nonslicing VLSI floorplanning problem. The algorithm is a hybrid simulated annealing (HSA) algorithm. First, the HSA uses a new constructive method to generate an initial B*-tree. Then, after a local search method in the HSA to exploit local information in the search region, we add a new operation on the B*-tree by swapping two subtrees in the B*-tree to generate a new configuration. At last, to balance the global exploration and local exploitation, we use a novel bias search strategy to determine if the new configuration produced by the operation on the B*-tree could be accepted. The HSA has been implemented and tested on the MCNC benchmarks [29]. Experimental results show that it can quickly produce optimal or nearly optimal solutions for the benchmark problems.

The paper is organized as follows. Section II is the problem statement. Section III reviews the B*-tree representation. The HSA is discussed in detail in Section IV, and empirical studies on the HSA are presented in Section V. Finally, conclusions are made in Section VI.

II. PROBLEM STATEMENT

Given a rectangular floorplanning region P with width W and height H (see Fig. 1), a set of modules $M = \{r_1, r_2, r_3, \dots, r_m\}$, in which module r_i is a rectangular block with fixed width w_i and height h_i , and given a netlist N specifying interconnections between the modules in M , the problem

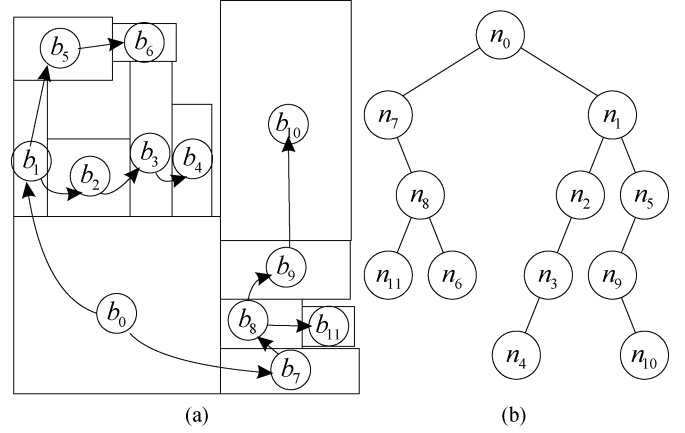


Fig. 2. (a) Admissible floorplan. (b) (horizontal) B*-tree representing the floorplan in (a).

is to pack all the modules into the rectangular region P, such that they meet the following conditions:

- 1) each module lies parallel to the coordinate axes;
- 2) no module overlaps with another;
- 3) all modules must be in the rectangular frame P;
- 4) a module may be rotated by 90° .

Given M and N , the objective of the floorplanning problem is to find a floorplan F such that a cost function is minimized.

In this paper, the cost of a floorplan F , $cost(F)$, consists of two parts: one is the area, $area(F)$, which is measured by the smallest rectangle that encloses the floorplan; the other is the interconnection cost between modules, $wirelength(F)$, which is the wirelength of all the nets specified by N , here the wirelength of a net is calculated by the half perimeter of the minimal rectangle, which encloses the centers of the modules connected by the net. More specifically, the cost function is defined as follows:

$$cost(F) = w \times \frac{area(F)}{norm_area} + (1 - w) \times \frac{wirelength(F)}{norm_wire} \quad (1)$$

where w is a weight, $w \in [0, 1]$, $norm_area$ and $norm_wire$ are the minimal area and the minimal interconnection cost of the problem, respectively. This kind of cost function has been adopted from [4], [21]. Since we do not know $norm_area$ and $norm_wire$ in practice, estimated values are used. In this paper, $norm_area$ and $norm_wire$ are set to $area(F)$ and $wirelength(F)$ of the initial floorplan in the HSA.

III. B*-TREE

A. B*-tree Representation

A floorplan with m rectangular modules can be represented by a horizontal B*-tree with m nodes (see Fig. 2). It is an ordered binary tree, with root corresponding to the module on the BL corner of the floorplan. Similar to the depth first search procedure [3], a B*-tree T which represents an admissible floorplan F is constructed in a recursive fashion: starting from the root, recursively the left subtree is constructed and then the right subtree is constructed. Let B_i denote the set of modules located

on the right-hand side of and adjacent to the module b_i , and n_i denote the node in the B*-tree corresponding to b_i . The left child of node n_i corresponds to the lowest module in B_i that is unvisited. The right child of n_i represents the module located above and adjacent to b_i , with its x -coordinate equal to that of b_i , and y -coordinate less than that of the top boundary of the module on the left-hand side of and adjacent to b_i , if any.

Since the root of the B*-tree represents the BL module of the floorplan F , the x - and y -coordinates of the module are $(x_{\text{root}}, y_{\text{root}}) = (0, 0)$. The B*-tree keeps the geometric relationship between two modules as follows. If node n_j is the left child of node n_i , then module b_j must be on the right-hand side of and adjacent to module b_i in the admissible floorplan, i.e., $x_j = x_i + w_i$. Besides, if node n_j is the right child of n_i , then module b_j must be above and adjacent to module b_i , with the x -coordinate of b_j equal to that of b_i , i.e., $x_j = x_i$. For an intuitive understanding of B*-tree, see Fig. 2.

For an admissible floorplan, we can construct a unique B*-tree, and vice versa [3]. A floorplan is admissible if no module can be shifted left or bottom without moving other modules. Given a feasible floorplan (no module overlaps with another), we can derive an admissible floorplan by compacting the modules to the left and bottom boundaries of the floorplan. In this paper, the search space of the VLSI floorplanning problem is limited to admissible floorplans.

B. Operations on B*-tree

In [3], the authors perturbed a B*-tree (a feasible solution) to another B*-tree using the following three operations:

- 1) rotate a module by 90° ;
- 2) move a module to another place;
- 3) swap two modules.

If a module is rotated by 90° , then the height and width of this module are swapped. Thus, if the height of this module is not equal to its width, then after rotating, the x - and y -coordinates of each module behind and above this module must be changed, and we get a new B*-tree. All the three operations can be done in $O(h)$ time, where h is the height of the B*-tree.

IV. HYBRID SIMULATED ANNEALING ALGORITHM

A. Initial B*-tree

To generate an initial floorplan, we define a new heuristic function for every module in this section, and pack the modules into the floorplanning region according to the heuristic values. The effect of the generated initial floorplan is demonstrated in Section V.

In practical VLSI floorplanning, modules with large areas or long lengths always receive priority consideration. Different from [2], [25], in order to consider both the factors of length and area, we define an evaluation function V_i for each rectangular module i as

$$V_i = \lambda_1 \times \frac{h_i \times w_i}{H \times W} + \lambda_2 \times \frac{h_i + w_i}{H + W} \quad (2)$$

where, h_i and w_i are the height and width of module i , respectively; H, W are the height and width of the floorplanning region

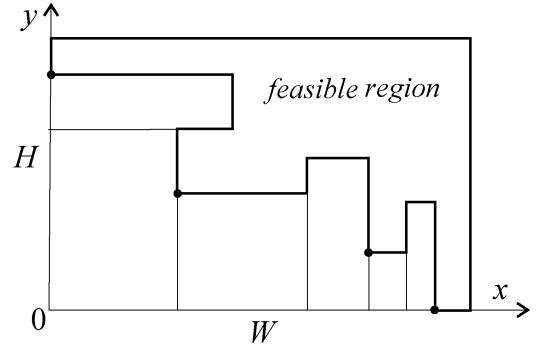


Fig. 3. Feasible region and points.

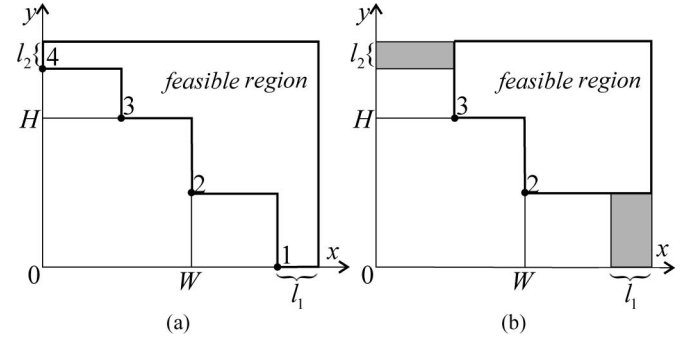


Fig. 4. (a) Feasible region. (b) Changed feasible region.

P , respectively; λ_1, λ_2 are non-negative weights, $\lambda_1 + \lambda_2 = 1$. In the HSA, we set $\lambda_1 = \lambda_2 = 0.5$ during the construction of an initial floorplan, which means that both the semiperimeter and area of a module contribute the same to the value of V_i .

A module with a larger value of V_i will be packed earlier, that is to say, it has a higher priority to be packed. Hence, we order the modules decreasingly with respect to the value of V_i , and the module set is given by $M' = \{r'_1, r'_2, r'_3, \dots, r'_m\}$.

The first module r'_1 is packed with its BL corner at the origin. Let B be the set of modules having been packed into the region. The remaining modules must be packed only in the feasible region where no module in B can be overlapped, and the border of the floorplanning region cannot be overstepped. Fig. 3 shows the feasible region (the area enclosed by the black lines), in which the remaining modules can be packed.

When adding a new module, we must find a point such that the new module can be packed in the feasible region with its BL corner at this point.

It must be noted that, we only consider those points (black points in Fig. 3) where the border line of the feasible region goes from top to bottom and then from left to right. These points are easy to identify, since with the B*-tree representation, we can find these points in linear time. If the distance between a point and the feasible region's border is less than the length of the smallest edge of unpacked modules, this point is called a bad point, since none of the unpacked modules can be packed with its BL corner at this point. We can discard the bad points and make some changes to the feasible region. For example, in Fig. 4(a), if both l_1 and l_2 are smaller than the smallest edge of

Algorithm 1: Initial B*-tree

```

begin
   $M' = \{r'_1, r'_2, r'_3, \dots, r'_m\}$ ;
  B*-tree  $B = \Phi$ ;
  While  $M'$  is not empty do
    use the way described in 1), 2), 3) to select a point and a module;
    let  $r'_i$  be the selected module, and pack  $r'_i$  at the selected point;
    add module  $r'_i$  to  $B$ ; remove module  $r'_i$  from  $M'$ ;
  end while
  return B*-tree  $B$ 
end

```

Fig. 5. Initial B*-tree algorithm.

unpacked modules, then we discard the bad points 1 and 4, and the changed feasible region is shown in Fig. 4(b).

Thus, we can describe the way of adding a module to B as follows:

- 1) find the points in the feasible region, sort these points in ascending order according to their y -coordinates, and choose the first point;
- 2) following the given order of modules, select the first module that can be packed in the feasible region with its BL corner at this point, include this module to the module set B ;
- 3) if no module can be packed at this point (if this point is a bad point, make changes to the feasible region), then discard this point, select the next point, go to 2).

Using the strategy of packing modules described above, the pseudocode of initial B*-tree algorithm can be described in Fig. 5.

B. Local Search Method in the HSA

This section presents a local search method, which will be used in the HSA. The local search method is based on the inner loop execution of simulated annealing algorithm in [3] and [26].

Given an initial floorplan encoded in a B*-tree, at each temperature, the local search method finds a local optimal solution through systematically examining those B*-trees obtained by rotating a module by 90° , moving a module to another place, or swapping two modules. If one of the operations leads to a floorplan with smaller cost or cost meeting an acceptance rate, the floorplan is accepted. The pseudocode of local search algorithm is presented in Fig. 6.

In the algorithm, m is the number of modules, k , $N_{\max}$ are parameters used to control the number of the three operations executed, mt is the number of the three operations attempted so far in this local search, $uphill$ is the number of new configurations accepted but not lead to a smaller cost, $reject$ is the number of configurations that are not accepted. The value of T will be presented in Section IV-E and $localbest$ denotes the best floorplan in this local search. The $reject_rate$ will be used in our HSA as a stopping criterion.

In our HSA, we have to store all costs generated during one run of the local search method, which will be used for the strategy to bias the search and change the temperature (see Section IV-E). Hence, we add these costs to a list.

Algorithm 2: The local search algorithm

```

begin
   $I$  is an initial configuration;
   $mt = 0$ ,  $uphill = 0$ ,  $reject = 0$ ;
   $N_{\max} = k * m$ ;
  an empty list  $LIST$ ;
  while  $uphill < N_{\max}$  and  $mt < 2 * N_{\max}$  do
    randomly use one of the three operations to generate a new
    configuration  $J$ ;
    calculate  $cost(J)$ , and add the cost to the list  $LIST$ ;
     $mt = mt + 1$ ;
     $\Delta C = cost(J) - cost(I)$ ;
    if  $\Delta C \leq 0$  then  $I = J$ ;  $localbest = J$ ;
    else
      randomly generate a number  $t$  between 0 and 1;
      if  $t < e^{\Delta C/T}$  then  $I = J$ ;  $uphill = uphill + 1$ ;
      else
         $reject = reject + 1$ ;
      end if
    end if
  end while
   $reject\_rate = reject / mt$ ;
  return configuration  $I$ 
  return  $reject\_rate$ 
  return  $localbest$ 
end

```

Fig. 6. Local search algorithm.

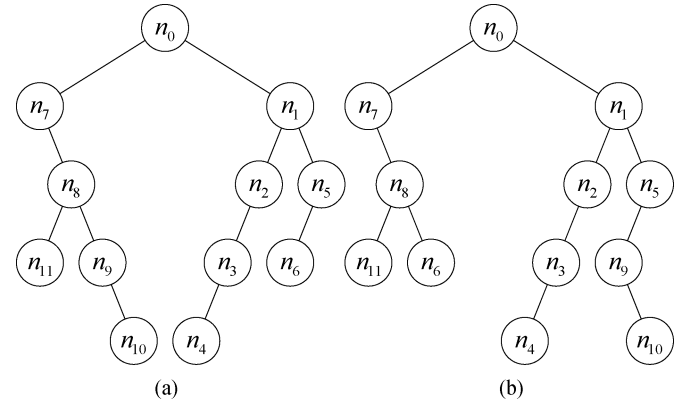


Fig. 7. New operation of swapping two subtrees.

C. A New Operation on the B*-tree

The local search approach, presented in Section IV-B only moves single module, or swaps two modules, in a B*-tree. In order to move more modules in each iteration for global exploration, in this paper, we develop a new operation by swapping two subtrees in a B*-tree. It is explained as follows:

- 1) randomly select an inner node n_i (not a leaf node) in the B*-tree;
- 2) randomly select a node n_j with the same tree height as n_i ;
- 3) swap the subtree rooted at n_i and the subtree rooted at n_j .

Fig. 7 shows an example of swapping two subtrees. If nodes n_9 and n_6 in Fig. 7(a) are selected, we swap the subtrees rooted at n_9 and n_6 , and the result is shown in Fig. 7(b). Note that swapping two leaves is equal to swapping two modules, which is one of the operations on the B*-tree in Section III. Hence, this new operation does not select two leaves in the B*-tree.

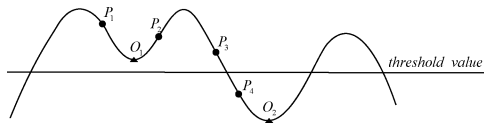


Fig. 8. Strategy to bias the search.

Moreover, in order to keep the B*-tree balanced, we select a node in 2) with the same tree height as that in 1).

Since searching a node in a tree needs $O(h)$ time, where h is the height of the tree, the operation of swapping two subtrees in the B*-tree can be made in $O(h)$ time.

D. Strategy to Bias the Search

In the HSA, we use a local search method to exploit local information in the search region, and a new operation on the B*-tree to explore the search space. A technique that is used to balance the exploration and exploitation of the solution space is using a bias search strategy [21], [27], [28]. Hence, we use the idea of the strategy to bias the search. After the local search in Section IV-B, we use the new operation on the B*-tree in Section IV-C to generate a new configuration. If the cost value of this configuration is equal to or less than a threshold value, then the new configuration is accepted as the current configuration, and used as an initial configuration for the next local search.

The threshold value is dynamic, which is dependent on the number of elements in the cost list after each local search. Suppose that there are 100 elements in the cost list after a local search. The threshold value is set to a value such that cost values of 100μ elements are greater than the threshold value, and cost values of $100(1 - \mu)$ elements are less than or equal to the threshold value, where μ is a ratio, $0 \leq \mu \leq 1$. This kind of threshold value is different from that in [21], where it is a static one. The basic idea behind this bias search strategy is illustrated in Fig. 8.

In Fig. 8, the curve depicts the cost landscape of a VLSI floorplanning problem (the objective of the floorplanning problem is to find a floorplan such that the cost is minimized). The points O_1 and O_2 are two local optima, point P_2 is the current configuration, and P_1 , P_3 , and P_4 are three points generated by the operation of swapping two subtrees. The horizontal line denotes a threshold value. Since the cost values of P_1 and P_3 are greater than the threshold value, the local search is not applied from them, but only applied from P_4 .

Using this bias search strategy, the HSA does not apply the local search from all the configurations generated by the new operation on the B*-tree, but just apply from some promising configurations. To explain this, in Fig. 8, if the local search is applied from P_1 , the local search will find the optimum O_1 , which is poor. If the local search is applied from P_3 , the local search will take more time than applied from P_4 to find the optimum O_2 . Therefore, our HSA only applies the local search from points with cost values equal to or less than the threshold value.

Algorithm 3: The HSA

```

begin
  Initial B*-tree  $B$  (by algorithm 1);
   $best = B$ ;  $I = B$ ;  $count = 0$ ;  $conv\_rate = 1$ ;
  an initial temperature  $T$ ;  $min$ ;
  while  $reject\_rate < conv\_rate$  and  $actual\_T > term\_T$  do
     $count = count + 1$ ;
    optimize  $I$  using the local search method (by Algorithm 2);
    if  $cost(localbest) < cost(best)$  then  $best = localbest$ ;
  end if
  update  $T$  and  $actual\_T$ ;
  if  $count > min$  then  $conv\_rate = 0.95$ ;
  end if
  sort all costs in the list  $LIST$  of the latest local search, use  $\mu$  to
  calculate the threshold value;
  swap two sub-trees on the return  $I$  (generated by Algorithm 2) to
  generate a new configuration  $Q$ ;
  if  $cost(Q) \leq threshold\ value$  then  $I = Q$ ;
    if  $cost(Q) < cost(best)$  then  $best = Q$ ;
  end if
  end if
end while
end

```

Fig. 9. Hybrid simulated annealing algorithm.

The value of μ determines the threshold value, which will balance the global exploration and local exploitation. When the value of μ is too small, most of the configurations generated by swapping two subtrees will be accepted, and a lot of local searches will be performed from unpromising configurations. In this case, the algorithm is time consuming. When the value of μ is too large, most of the configurations generated by swapping two subtrees will be rejected. In this case, some promising search points are ignored, and the probability of finding a good solution is low. Therefore, it is important to take an appropriate value of μ . An empirical study of μ and the effectiveness of the threshold strategy can be found in Section V.

E. Description of the HSA

Initially, the HSA generates an initial B*-tree using the technique described in Algorithm 1, Section IV-A. Then, the HSA uses the local search algorithm, Algorithm 2, Section IV-B, to optimize a configuration, and uses the new operation in Section IV-C on the output of the local search to generate a new configuration. If the cost of this configuration is not higher than the threshold value, then this configuration is accepted as an initial configuration for the next local search. This process is repeated until the algorithm stops. In the HSA, in order to avoid reaching convergence or the terminating temperature too quickly, we set an integer min to control the least number of local searches performed. Based on the aforementioned discussion, the pseudocode of the HSA can be described in Fig. 9.

In the HSA, $conv_rate$ is the convergence rate of our HSA. The value of $conv_rate$ is the same as that in [3]; $term_T$ is the terminating temperature; min is a constant which will be given in the following section. The algorithm will stop when $reject_rate \geq conv_rate$, or the temperature is cool enough ($actual_T \leq term_T$).

```

Update  $T$  and  $actual\_T$ :
begin
   $r\_t = e^{lamda * T / sv}$ ;
   $T = r\_t * T$ ;
  if  $min == count$  then
     $T = estimate\_avg / \ln(p)$ ;
     $T = T * p^{min}$ ;
     $actual\_T = e^{estimate\_avg / T}$ ;
  end if
  if  $min > count$  then
     $actual\_T = e^{estimate\_avg / T}$ ;
  end if
end

```

Fig. 10. Update T and $actual_T$.TABLE I
CHARACTERISTICS OF THE MCNC BENCHMARK PROBLEMS

benchmarks	modules	I/o pads	nets	pins	total area (mm^2)
apte	9	73	97	287	46.56
xerox	10	107	203	698	19.35
hp	11	43	83	309	8.83
ami33	33	42	123	522	1.16
ami49	49	24	408	953	35.45

The cooling schedule of the HSA is the same as that in a normal simulated annealing algorithm [3], [26]. In the HSA, the way to update T and $actual_T$ is described in Fig. 10.

In the algorithm, $lamda$, $estimate_avg$, and p are different constants, which are used to control the temperature. These values are given in the following section; sv is the standard deviation of all the costs in each local search (all costs in the list $LIST$), which has a great effect on $actual_T$. The initial value of T is set to $estimate_avg / \ln(p)$.

V. EMPIRICAL STUDIES ON THE HSA

In this section, we use the MCNC benchmark problems ([29], also see Table I) to empirically study the performance of the HSA. We also study the effect of constructing an initial B*-tree, the determination of the value of μ , the effect of the threshold strategy and the new operation, and the effect of weights assigned to the two optimization objectives in (1) on the performance of the HSA.

A. Performance of the HSA

In this part, two experiments are carried out for two different objectives as given in (1): area optimization, simultaneous area, and wavelength optimization. In the HSA, the values of λ_1 , λ_2 in constructing an initial B*-tree were both set to 0.5, the value of μ was set to 0.65 for all the tested problems since it was the best value found empirically, factor k was set to 300, $lamda$ was set to 1.3, p was set to 0.9, $estimate_avg$ was set to $5.3 * 10^{-4}$, $term_T$ was set to 0.1, min was set to 7. To study the performance of the HSA, we implemented the HSA in C++ programming language on a Pentium IV computer with 2.0 GHz CPU and 512MB RAM (Linux environment).

1) In this experiment, the objective is to minimize the floorplanning area. The weight w for the area and the interconnection

TABLE II
COMPARISON OF RESULTS ON MCNC BENCHMARK PROBLEMS BY THE LFF, BSA, MA, PSO, AND HSA

algorithms	apte	xerox	hp	ami33	ami49
LFF	47.70	20.12	9.10	1.21	37.30
BSA	46.92	19.93	9.31	1.23	36.68
MA	-	-	-	1.21	38.09
PSO	46.92	20.38	-	1.29	38.93
HSA	46.92	20.01	9.01	1.20	36.48

minimization objective was set to 1. We compare the results of our algorithm with those of the algorithms in [2], [3], [21], [30]. The algorithm in [2] less flexibility first (LFF) is based on the principle of less flexibility first. The algorithm in [3] BSA is based on the simulated annealing method with the B*-tree representation. The algorithm in [21] is an MA for the nonslicing and hard-module VLSI floorplanning problems. The algorithm in [30] PSO is based on the particle swarm optimization technique. Note that, the results obtained by LFF, simulated annealing method with the B*-tree representation (BSA), and MA are only optimizing the floorplanning area. In this paper, the BSA and the HSA were implemented in the same programming language on the same platform.

The experimental results of LFF, MA, and particle Swarm optimization (PSO) are directly taken from [2], [21], [30], respectively. The MA was carried out on a Pentium IV computer with 2.0 GHz CPU and 512MB RAM; the configuration is the same as ours. The LFF was run on a 250 MB Sun Spark station. The PSO was implemented in MATLAB 6.5 on a computer with an AMD CPU running at 1.46 GHz and with 512 MB RAM.

We tested the BSA and the HSA on each benchmark problem 100 times, recorded the results obtained from each run, and took the average. The results of the average area (millimeter square) of LFF, BSA, MA, PSO, and HSA are shown in Table II, where “-” means that the results are not reported by the corresponding algorithm.

The experimental results obtained using the MCNC benchmarks show that the HSA has 1.71%, 1.00%, 0.55%, 0.83%, and 2.25% improvement on the area, respectively, over the LFF; has 0.04%, -0.40%, 3.33%, 2.50%, and 0.33% improvement on the area, respectively, over the BSA; has 0.83% and 4.41% improvement on the area over the MA on ami33, ami49; has 0.04%, 1.85%, 7.50%, and 6.72% improvement on the area over the PSO on apte, xerox, ami33, and ami49.

Since the LFF, MA, and PSO were implemented in different programming languages on different platforms, and the computation time of PSO was not reported in [30], we have not listed the computation time of each algorithm in Table II. The computation time of HSA on each benchmark problem was reported in Table IV. The runtime of MA (the computer configuration is the same as ours) on ami33, ami49 was set to 1800 s. For all the benchmark problems, the maximum computation time of BSA was 6.59 s, and it was 6.78 s for HSA. But the average computation time used by HSA is 3.334 s, which is less than 3.602 s by BSA. Considering the experimental results in Table II and the computation time used by each algorithm, it shows that the

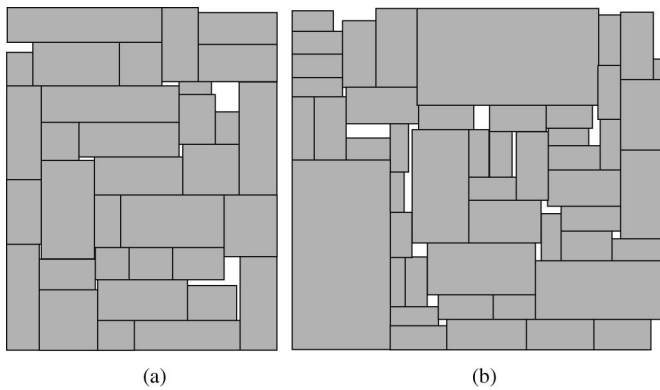


Fig. 11. (a) Ami33 floorplan example. (b) Ami49 floorplan example.

HSA can quickly produce optimal or nearly optimal solutions for the MCNC tested problems.

Fig. 11(a) shows the floorplan obtained using our algorithm on the example ami33. Its area is 1.19 mm^2 (its area usage is 97.5%), and its wirelength is 121.1 mm. Fig. 11(b) shows the floorplan obtained using our algorithm on the example ami49. Its area is 36.39 mm^2 (its area usage is 97.4%), and its wirelength is 1703.9 mm.

2) In this experiment, the area and total wirelength are optimized simultaneously. It must be pointed out that although the optimization problem has two objectives, multiobjective optimization techniques may not be suitable for it as the two objectives are not equally important, and the weights assigned to the two objectives should be controllable by the designer to meet other floorplanning requirements.

The weight w for the area and the interconnection minimization objective was set to 0.5 (as in [4]). We compare our algorithm with the algorithms in [4], [6], [30]. The algorithm in [4] (O-tree) is based on a deterministic algorithm with the O-tree representation, and it was performed on a 200 MHz SUN Sparc Ultra-I workstation with 512 MB memory. The algorithm in [6] is based on the simulated annealing method with the TCG representation, and it was carried out on a 433MHz SUN Sparc Ultra-60 workstation with 1GB memory. The configuration of PSO has been described earlier.

We tested the HSA on each benchmark problem 100 times, recorded the results obtained from each run, and took the best/average. The results of area (millimeter square) and wirelength (millimeter) by O-tree, TCG, PSO, and HSA are shown in Table III, and “-” means that the results are not reported by the corresponding algorithm. It should be noted that the PSO in [30] was tested on each benchmark problem only five times. The experimental results of O-tree, TCG and PSO are directly taken from [4], [6] and [30], respectively.

From Table III, it can be seen that, with the same w in O-tree, our HSA obtains better average areas than O-tree for all the five benchmarks, but has a little difference on average wirelengths. It must be noted that for the five benchmark problems, all the optimal costs of our algorithm are smaller than those by O-tree. Furthermore, in terms of area utilization and wirelength, while the HSA obtains better areas or similar areas compared

with the other algorithms, the wirelengths are a little worse. It probably results from the fact that the weighted sum method of multiobjective optimization has a difficulty in weight assignment. Moreover, the value of w also has a strong impact on the performance of the HSA. In this experiment, the maximum computation time of the HSA is 35.43 s.

B. Effect of Constructing an Initial B*-tree

For describing the effect of constructing an initial B*-tree, we directly list in Table IV the average areas of the MCNC benchmark problems after constructing an initial B*-tree [A], the average areas of HSA without constructing an initial B*-tree (using randomly generated initial floorplans) [B], and the average areas of HSA [C]. All these results are obtained by averaging on the outputs of 100 runs of the algorithms.

It can be seen from Table IV that the results in [A] are similar to those obtained by some of the algorithms in Table II. Except apte, the results in [C] are better than those in [B]. Hence, it is obvious that our way of constructing an initial B*-tree is effective, and produces good initial solutions for the local search. Therefore, although the idea of constructing an initial B*-tree seems simple, it allows modules to be placed more compactly.

C. Determination of the Value of μ

The value of μ is very important since it balances exploration and exploitation in the HSA, which affects the computation time and the quality of solutions. To determine an appropriate value of μ , we tested the HSA with different values, i.e., $\mu = 0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 1.00$, on each benchmark problem 100 times, recorded the results obtained from each run, and took the average.

Note that, when the value of μ is too small, the quality of the optimal solutions by the HSA cannot be guaranteed, and the computation time is long. The reason is that most of the solutions generated by swapping two subtrees in a B*-tree are accepted, and a lot of local searches are performed. Moreover, when the value of μ is too large, most of the solutions generated by swapping two subtrees in a B*-tree are rejected, and the probability of finding a good solution is low. Especially, if μ is equal to 1, the strategy has no effect on HSA. Hence, in Table V, we only present the results for the values of μ ranging from 0.4 to 0.90.

There are three observations that can be drawn from Table V:

- 1) when μ is equal to 0.55, 0.60, or 0.65, the algorithm finds better optimal solutions for all the benchmark problems than the other values of μ ;
- 2) when μ is equal to 0.60, 0.65, or 0.70, the algorithm uses less computation times;
- 3) when μ is equal to 0.65, the algorithm takes a good balance between the obtained optimal values and computation times.

The three observations suggest that the value of μ should be set to 0.65 for all the benchmark problems. By doing this, the HSA has a good tradeoff between computation time and optimality of solutions.

TABLE III
COMPARISON OF RESULTS ON MCNC BENCHMARK PROBLEMS BY THE O-TREE, TCG, PSO, AND HSA

algorithms	apte		xerox		hp		ami33		ami49	
	best (area/wire)	average (area/wire)	best (area/wire)	average (area/wire)	best (area/wire)	average (area/wire)	best (area/wire)	average (area/wire)	best (area/wire)	average (area/wire)
O-tree	- / -	53.2/370	- / -	22.4/447	- / -	10.5/167	- / -	1.34/59.8	- / -	42.0/777
TCG	- / -	48.48/378.0	- / -	22.42/385.0	- / -	9.49/151.8	- / -	1.24/50.29	- / -	38.20/663.1
PSO	47.31/263	- / -	20.2/477	- / -	9.5/136	- / -	1.28/69	- / -	38.8/880	- / -
HSA	47.12/480.0	48.12/478.9	20.89/513	21.86/482.0	9.47/144.3	9.43/214.3	1.21/61.2	1.25/59.8	37.80/1020.3	40.01/899.0

TABLE IV
EFFECT OF CONSTRUCTING AN INITIAL B*-TREE

algorithms	apte		xerox		hp		ami33		ami49	
	area	time	area	time	area	time	area	time	area	time
A	47.97	0.00	20.54	0.00	9.90	0.00	1.29	0.01	38.25	0.01
B	46.90	0.45	20.02	1.23	9.02	0.89	1.20	5.60	36.50	11.00
C	46.90	3.95	20.01	0.57	9.01	0.27	1.20	5.10	36.48	6.78

TABLE V
TEST RESULTS ON THE PERFORMANCE OF HSA WITH DIFFERENT VALUES OF μ

μ	apte		xerox		hp		ami33		ami49	
	area	time	area	time	area	time	area	time	area	time
0.40	46.92	4.70	20.03	0.85	9.04	0.17	1.25	3.03	36.68	13.50
0.50	46.91	2.26	20.04	0.45	9.00	3.02	1.24	4.67	36.57	9.00
0.55	46.89	4.03	19.88	0.92	8.99	1.26	1.24	4.83	36.59	8.75
0.60	46.90	3.80	20.02	0.48	9.00	0.83	1.21	5.00	36.50	6.42
0.65	46.90	3.95	20.01	0.57	9.01	0.27	1.20	5.10	36.48	6.78
0.70	46.89	5.86	20.08	0.46	9.03	0.16	1.20	7.20	36.49	6.56
0.75	46.91	5.01	20.03	1.32	9.00	5.99	1.21	4.10	36.48	4.95
0.80	46.92	6.00	19.89	6.01	8.99	6.83	1.25	4.58	36.70	11.00
0.90	46.91	6.53	19.85	9.04	8.97	9.95	1.26	6.75	36.78	15.23

TABLE VI
EFFECT OF THE THRESHOLD STRATEGY AND THE NEW OPERATION

algorithms	apte		xerox		hp		ami33		ami49	
	area	time	area	time	area	time	area	time	area	time
D	46.92	6.99	20.01	8.64	8.99	10.03	1.27	9.08	36.80	13.33
E	46.90	3.95	20.01	0.57	9.01	0.27	1.20	5.10	36.48	6.78

TABLE VII
TEST RESULTS OBTAINED BY THE HSA WITH DIFFERENT WEIGHTS

weights	apte		xerox		hp		ami33		ami49	
	area	time	area	time	area	time	area	time	area	time
$w = 1$	46.90	1271.9	20.01	738.9	9.01	240.1	1.20	117.5	36.48	1907.0
$w = 0.5$	48.12	468.9	21.86	482.0	9.43	214.3	1.25	59.8	40.01	899.0
$w = 0$	52.80	348.0	22.32	353.2	11.20	120.0	1.41	43.8	43.82	580.3

D. Effect of the Threshold Strategy and the New Operation

For assessing the effect of the threshold strategy and the new operation, we tested the HSA with and without the threshold strategy and the new operation on each benchmark 100 times, and list in Table VI the average areas and computation times of the HSA with [E] and without [D] the threshold strategy and the new operation.

It can be seen from Table VI that [E] takes less computation time than [D] for all the tested benchmark problems, and the optimal values are comparable. Therefore, using the threshold strategy and the new operation, the HSA performs faster in finding optimal or nearly optimal solutions. The effect of the

threshold strategy and the new operation on the B*-tree are significant.

E. Tradeoff Between Two Optimization Objectives

To study the tradeoff between the two optimization objectives, i.e., the minimal area and minimal interconnection, we tested the HSA on the benchmark problems with three different configurations of weights, i.e., $w = 1$, $w = 0.5$, and $w = 0.0$. We ran the HSA on each benchmark problem 100 times and recorded the test results. Table VII shows the test results of average area (millimeter square) and average wirelength (millimeter).

With weight $w = 1$, the HSA focuses on minimizing the area, and the results on area are lower, but the results on wirelength are higher, than those with weights $w = 0.5$ and $w = 0.0$. With weight $w = 0.0$, our HSA focuses on minimizing the wirelength, and the results on area are not good. Therefore, the results in Table VII indicate that the HSA is sensitive to the configuration of the weight w . The optimal configuration of the weight w may be different for different problems, which needs further investigation.

VI. CONCLUSION

This paper has presented a hybrid simulated annealing algorithm for the nonslicing VLSI floorplanning problem. The algorithm is based on a new greedy method to construct an initial B*-tree, a new operation on the B*-tree by swapping two subtrees to explore the search space, and a dynamic bias search strategy to balance global exploration in the search space and exploitation in the local search region. Experimental results using the MCNC benchmarks show that the HSA outperforms some known algorithms for the nonslicing floorplanning problem. Since the floorplanning problem usually has several objective functions, improving our HSA for a multiobjective floorplanning problem is a topic needing further investigation.

ACKNOWLEDGMENT

The authors would like to thank Prof. Y. W. Chang (National University of Taiwan) for providing us with the B*-tree package. Special thanks also go to the editors and anonymous reviewers for their valuable comments, which help improving the quality of our manuscript greatly.

REFERENCES

- [1] C. J. Alpert and D. P. Mehta, *Handbook of Algorithm for Physical Design Automation*. New York: Auerbach Publications, 2008, pp. 139–142.
- [2] S. Dong, X. Hong, Y. Wu, and Y. Lin, “VLSI block placement using less flexibility first principles,” in *Proc. Asia and South Pacific Design Automation Conf.*, 2001, pp. 601–604.
- [3] Y. C. Chang, Y. W. Chang, G. M. Wu, and S. W. Wu, “B*-trees: a new representation for non-slicing floorplans,” in *Proc. 37th ACM/IEEE Annual Design Automation Conf.*, 2000, pp. 458–463.
- [4] P. N. Guo, T. Takahashi, C. K. Cheng, and T. Yoshimura, “An O-tree representation of non-slicing floorplanning and its application,” in *Proc. 36th ACM/IEEE Annual Design Automation Conf.*, 1999, pp. 268–273.
- [5] X. Hong, G. Huang, Y. Cai, J. Gu, S. Dong, C. Chen, and J. Gu, “Corner block list: An effective and efficient topological representation of non-slicing floorplan,” in *Proc. Int. Conf. on Computer Aided Design*, Nov. 2000, pp. 8–13.
- [6] J. M. Lin and Y. W. Chang, “TCG: A transitive closure graph-based representation for non-slicing floorplans,” *IEEE Trans. Very Large Scale Integ. Syst.*, vol. 13, no. 2, pp. 288–292, Feb. 2005.
- [7] J. M. Lin and Y. W. Chang, “TCG-S: Orthogonal coupling of P*-admissible representations for general floorplans,” *IEEE Trans. CAD*, vol. 23, no. 6, pp. 968–980, Jun. 2004.
- [8] X. Tang and D. F. Wong, “FAST-SP: A fast algorithm for block placement based on sequence pair,” in *Proc. Asia and South Pacific Design Automation Conf.*, Dec. 2001, pp. 521–526.
- [9] E. F. Y. Young, C. C. N. Chu, and Z. C. Shen, “Twin binary sequences: A nonredundant representation for general non-slicing floorplan,” *IEEE Trans. CAD*, vol. 22, no. 4, pp. 457–469, Apr. 2003.
- [10] S. Zhou, S. Dong, C. K. Cheng, and J. Gu, “ECBL: An extended corner block list with solution space including optimum placement,” in *Proc. Int. Symposium on Physical Design*, Apr. 2001, pp. 150–155.
- [11] H. Zhou and J. Wang, “ACG-adjacent constraint graph for general floorplans,” in *Proc. IEEE Conf. on Computer Design*, 2004, pp. 572–575.
- [12] C. Zhuang, Y. Kajitani, K. Sakanushi, and L. Jin, “An enhanced Q-sequence augmented with empty-room-insertion and parenthesis trees,” in *Proc. Design, Automation and Test in Europe Conf. and Exhib.*, 2002, pp. 61–68.
- [13] H. H. Chan, S. N. Adya, and I. L. Markov, “Are floorplan representations important in digital design?” in *Proc. Int. Symposium on Physical Design*, pp. 168–173, 2000.
- [14] S. B. Brenda, Jr., G. C. Edward, and L. R. Ronald, “Orthogonal packings in two dimensions,” *SIAM J. Comput.*, vol. 9, no. 4, pp. 808–826, 1980.
- [15] C. Bernard, “The bottom-left bin packing heuristic: An efficient implementation,” *IEEE Trans. Comput.*, vol. 32, no. 8, pp. 697–707, Aug. 1983.
- [16] D. Q. Liu and H. F. Teng, “An improved BL-algorithm for genetic algorithms of the orthogonal packing of rectangles,” *Eur. J. Oper. Res.*, vol. 112, no. 2, pp. 413–419, Jan. 1999.
- [17] E. K. Burke, G. Kendall, and G. Whitwell, “A new placement heuristic for the orthogonal stock-cutting problem,” *Oper. Res.*, vol. 52, no. 4, pp. 655–671, Jul. 2004.
- [18] T. Lengauer and R. Muller, “A robust framework for hierarchical floorplanning with integrated global wiring,” in *Proc. Int. Conf. Computer Aided Design*, 1991, pp. 148–151.
- [19] W. Huang, D. Chen, and R. Xu, “A new heuristic algorithm for rectangle packing,” *Computers Oper. Res.*, vol. 34, no. 11, pp. 3270–3280, Nov. 2007.
- [20] C. Valenzuela and P. Wang, “VLSI placement and area optimization using a genetic algorithm to breed normalized postfix expressions,” *IEEE Trans. Evol. Comput.*, vol. 6, no. 4, pp. 390–401, Aug. 2002.
- [21] M. Tang and X. Yao, “A memetic algorithm for VLSI floorplanning,” *IEEE Trans. Syst., Man, and Cybern., Part B*, vol. 37, no. 1, pp. 62–69, Feb. 2007.
- [22] C. Sechen and A. L. Sangiovanni-Vincentelli, “The timber wolf placement and routing package,” *IEEE J. Solid-State Circuits*, vol. SC-20, no. 2, pp. 510–522, Apr. 1985.
- [23] S. Sur-Kolay, “Studies on nonslicible floorplans in VLSI layout design,” Ph.D. dissertation, Dept. Comput. Sci. Eng., Jadavpur Univ., Calcutta, 1991.
- [24] D. F. Wong and C. L. Liu, “Floorplan design for rectangular and L-shaped modules,” in *Proc. Int. Conf. Computer Aided Design*, pp. 520–523, Nov. 1987.
- [25] L. Wei, D. Zhang, and Q. Chen, “A least wasted first heuristic algorithm for the rectangular packing problem,” *Computers Oper. Res.*, vol. 36, no. 5, pp. 1608–1614, May 2009.
- [26] S. Kirkpatrick, C. Gelatt, and M. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [27] H. Ishibuchi and T. Murata, “A multi-objective genetic local search algorithm and its application to flowshop scheduling,” *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 28, no. 3, pp. 392–403, Aug. 1998.
- [28] K. H. Liang, X. Yao, and C. Newton, “Evolutionary search of approximated n-dimensional landscapes,” *Int. J. Knowledge-Based Intell. Eng. Syst.*, vol. 4, no. 3, pp. 172–183, Jul. 2000.
- [29] The MCNC Benchmark Problems for VLSI Floorplanning. [Online]. Available: <http://www.mcnc.org>
- [30] G. Chen, W. Guo, and Y. Chen, “A PSO-based intelligent decision algorithm for VLSI floorplanning,” *Soft Computing*, vol. 14, no. 12, pp. 1329–1337, Oct. 2009.



Jianli Chen was born in Xiamen, China, on October 15, 1985. He received the B.Sc. degree in information and computing sciences from College of Mathematics and Computer Science, Fuzhou University, Fuzhou, China, in 2007. Currently, he is working toward the Ph.D. degree at Center for Discrete Mathematics and Theoretical Computer Science, Fuzhou University, Fuzhou, China.

His research interests include VLSI floorplanning and placement, artificial intelligence, and intelligent computing.



Wenxing Zhu received the B.Sc. degree in applied mathematics, the M.Sc. and Ph.D. degrees in operations research and cybernetics, all from the Shanghai University, Shanghai, China, in 1989, 1992, and 1996, respectively.

Since 1996, he has been with Fuzhou University, where he is a Professor at Center for Discrete Mathematics and Theoretical Computer Science, Fuzhou University, Fuzhou, China since 2004. His research interests include optimization theory and applications, algorithms for VLSI design automation, and data mining. He is the author or coauthor of more than 65 papers in refereed journals, including *Journal of Computer and System Sciences*, *Computers and Operations Research*, *Journal of Global Optimization*, *Journal of Optimization Theory and Applications*, and *Optimization Methods and Software*.



M. M. Ali received the B.Sc. and M.Sc. degrees in mathematics from the Dhaka University, Dhaka, Bangladesh, in 1982 and 1983, respectively, and the Ph.D. degree in stochastic global optimization from the University of Technology, Loughborough, U.K., in 1994.

From 1994 to 1996, he held Research Fellow positions at Plymouth University, U.K., and Abo Akademi University, Turku, Finland. Since 1997, he has been with School of Computational and Applied Mathematics, Witwatersrand University, Johannesburg, South Africa, where he is a Professor since 2010. His research interests include the development of stochastic global optimization algorithms and their applications to real-life problems. He is the author or coauthor of more than 40 scientific papers in refereed international journals.